Unit 7, Lesson 1 Reference Sheet

Introduction to Databases & SQL with SQLite

Why Databases?

- **CSV files** store data persistently, but:
  - No built-in search, filter, or sort
  - Everything is a string (need manual type conversion)
  - Changing one row means rewriting the whole file
  - No aggregation (average, count, etc.)
- **Databases** solve these problems:
  - Built-in search, filter, sort, aggregation
  - Real data types (numbers, text, etc.)
  - Surgical updates and deletes
  - Structured storage with rules

SQLite – The Database We Use

- **Built into Python** (`import sqlite3`) – no extra installation
- **Single file** (`.db`) – like a CSV, but smarter
- **No server** – no passwords or setup
- **Used everywhere** (iPhone, Android, Chrome, etc.)

Our tool for this lesson:
https://sqliteonline.com/ – browser-based SQL editor

- Click **File** → **Open DB** to load a `.db` file
- Write SQL in the editor, click **Run**
- Save your database with **File** → **Save DB**

SQL Data Types (SQLite)

Type	Description	Python Equivalent
`TEXT`	String of characters	`str`
`INTEGER`	Whole number	`int`
`REAL`	Decimal number	`float`
`NULL`	Missing / unknown value	`None`

Column Modifiers (Rules)

Modifier	Meaning
`PRIMARY KEY`	Unique identifier for each row
`AUTOINCREMENT`	Automatically assign the next integer (1, 2, 3…) – used with `PRIMARY KEY`
`NOT NULL`	Column cannot be empty (required)

CREATE TABLE – Defining the Structure

```
CREATE TABLE IF NOT EXISTS table_name (
    column1_name type modifiers,
    column2_name type modifiers,
    ...
);
```

Example – movies table:

```sql
CREATE TABLE IF NOT EXISTS movies (
    id      INTEGER PRIMARY KEY AUTOINCREMENT,
    title   TEXT NOT NULL,
    genre   TEXT NOT NULL,
    rating  REAL,
    year    INTEGER,
    review  TEXT
);
```

- `IF NOT EXISTS` prevents an error if the table already exists.
- `id` is automatically assigned – never provide it in an `INSERT`.

INSERT INTO – Adding Data

```sql
INSERT INTO table_name (column1, column2, ...)
VALUES (value1, value2, ...);
```

Rules:

- Column names and values must match in number and order.
- Text values use **single quotes**: `'Action'`
- Numbers have **no quotes**: `9.0`
- The `id` column (if `AUTOINCREMENT`) is omitted – the database fills it.
- Optional columns (no `NOT NULL`) can be left out – they become `NULL`.

Example – adding a movie:

```sql
INSERT INTO movies (title, genre, rating, year, review)
VALUES ('The Dark Knight', 'Action', 9.0, 2008, 'Batman vs Joker masterpiece');
```

Common Mistakes (and How to Avoid Them)

Mistake	Why It's Wrong / Fix
Providing an `id` value	`AUTOINCREMENT` assigns it – leave `id` out of the column list.
Wrong number of values	List the same number of columns as values.
Numbers in quotes: `'9.0'`, `'2008'`	Without quotes they are numbers; with quotes they become text.
Skipping a `NOT NULL` column	That column is required – you must provide a value.
Using double quotes for text: `"Action"`	SQL uses **single quotes** for text.
Apostrophe inside text: `'Schindler's List'`	Double the apostrophe: `'Schindler''s List'`

Quick Reference – First Queries

```sql
-- View all columns from a table
SELECT * FROM movies;

-- View specific columns
SELECT title, rating FROM movies;

-- Filter rows
SELECT title, rating FROM movies WHERE genre = 'Action' AND rating > 8;

-- Sort results
SELECT title, rating FROM movies ORDER BY rating DESC;

-- Count rows
SELECT COUNT(*) FROM movies;

-- Average rating
SELECT AVG(rating) FROM movies;
```

Your First Practice Tasks

1. **Load** `movies.db` in SQLite Online.
2. **Run** `SELECT * FROM movies;` – see all movies.
3. **Create your own table** (e.g., `games`, `songs`):

```sql
CREATE TABLE IF NOT EXISTS my_movies (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    genre TEXT NOT NULL,
    rating REAL,
    year INTEGER,
    review TEXT
);
```

4. **Insert at least 5 movies** into your table:

```sql
INSERT INTO my_movies (title, genre, rating, year, review)
VALUES ('Your Title', 'Genre', 8.5, 2020, 'Your review');
```

5. **Verify** with `SELECT * FROM my_movies;`

Next Lesson Preview

Lesson 2: SELECT & WHERE – Querying Power

- Filtering with `WHERE`
- Sorting with `ORDER BY`
- Aggregates: `COUNT`, `AVG`, `MAX`, `MIN`, `SUM`
- Pattern matching with `LIKE`
- Limiting results with `LIMIT`

Keep this sheet handy – you'll use it throughout the unit!